

4

## Normalisation d'une BDR



## Notion de dépendance fonctionnelle

| LIVRE  |                   |                |               |            |      |
|--------|-------------------|----------------|---------------|------------|------|
| NUMERO | TITRE             | AUTEUR         | ISBN          | DATE_ACHAT | EMPL |
| 1029   | L'humanité perdue | Finkelkraut A. | 2 02 033300 7 | 14/10/2008 | F3   |
| 1030   | L'humanité perdue | Finkelkraut A. | 2 02 033300 7 | 14/10/2008 | F3   |
| 1032   | Mercure           | Nothomb A.     | 2 253 14911 X | 14/10/2008 | G5   |
| 1045   | Eva Luna          | Allende I.     | 2 253 05354 6 | 22/2/2009  | F3   |
| 1067   | Mercure           | Nothomb A.     | 2 253 14911 X | 24/2/2009  | G5   |
| 1022   | Mercure           | Nothomb A.     | 2 253 14911 X | 3/10/2008  | G6   |

- si deux lignes ont la même valeur de ISBN, alors elles ont aussi les mêmes valeurs de TITRE et AUTEUR

On dit que :

- il existe une dépendance fonctionnelle de ISBN vers TITRE et AUTEUR: **ISBN → TITRE, AUTEUR**
- ISBN détermine (est un déterminant de) TITRE et AUTEUR
- TITRE et AUTEUR (dépendent de) ou sont déterminés par ISBN



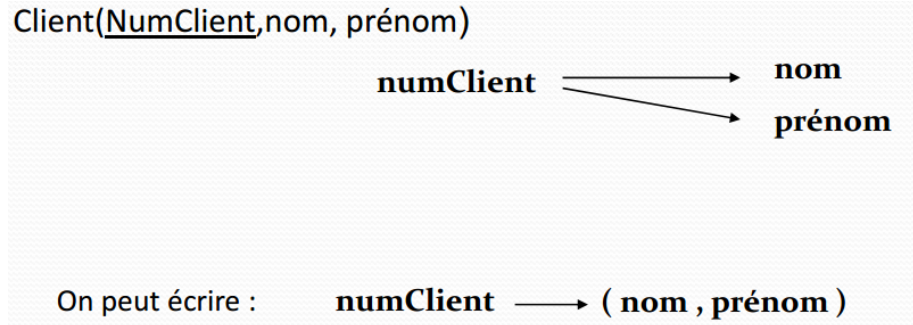
## Situation de redondance

- Par définition, un identifiant détermine toutes les colonnes de la table
- si un groupe de colonnes détermine chaque ligne de la table, il constitue par définition un identifiant de la table
- Il y a redondance interne dès qu'il existe un déterminant qui n'est pas un identifiant de la table
- Par exemple: ISBN est un déterminant dans LIVRE mais il n'en est pas un identifiant. Il entraîne donc des redondances internes.
- Une dépendance fonctionnelle dont le déterminant n'est pas un identifiant est dite anormale
- Une table qui est le siège d'une dépendance fonctionnelle anormale est dite non normalisée
- Une table sans dépendance fonctionnelle anormale est dite normalisée
- Décomposer une table de manière à éliminer ses dépendances anormales consiste à normaliser cette table
- **Il est essentiel que toutes les tables d'une base de données soient normalisées**



## Définition

- **Définition** : un attribut Y dépend fonctionnellement d'un attribut X si et seulement si une valeur de X détermine une et unique valeur de Y.
- **Notation** :  $X \rightarrow Y$  (X détermine Y ou Y dépend fonctionnellement de X)
- **Exemple** :  $\text{numClient} \rightarrow \text{nomClient}$  . Par contre :  $\text{nomClient} \rightarrow \text{numClient}$  est fausse car on peut avoir plusieurs clients de même nom mais de numClient différents.
- Tous les attributs d'une relation sont en dépendance fonctionnelle directe avec l'identifiant (clé primaire) de cette relation.





## Types de DFs

- XY → Z est **élémentaire** si ni X, ni Y pris individuellement ne déterminent Z.

### Exemples

- Nom, DateNaissance, LieuNaissance → Prénom est élémentaire.
- RefProduit, LibelleProduit → PU n'est pas élémentaire car il suffit d'avoir la référence du produit pour déterminer le prix unitaire.
- Une Dépendance fonctionnelle  $X \rightarrow Y$  est **canonique** si sa partie droite ne comporte qu'un seul attribut
- Un ensemble F de dépendances fonctionnelles est canonique si chacune de ses dépendances est canonique.



## Axiomes et règles d'inférence

- Des **axiomes** et des **règles d'inférence** permettent de découvrir de nouvelles dépendances à partir d'un ensemble initial. Dans ce qui suit nous considérons R une relation.
- Les propriétés: Réflexivité, Augmentation et Transitivité sont connues sous le nom « Axiomes d'Armstrong »



# Réflexivité et Augmentation

## Réflexivité

- Si  $Y \subseteq X$  Alors  $X \rightarrow Y$ .
- Tout ensemble d'attributs détermine lui-même ou une partie de lui-même
- $AB \rightarrow AB$ ;  $AB \rightarrow A$ ;  $AB \rightarrow B$

## Augmentation

- Si  $X \rightarrow Y$  alors  $XZ \rightarrow YZ$
- Si  $X$  détermine  $Y$ , les 2 ensembles d'attributs peuvent être enrichis par un même troisième
- **(Nom, Prenom)  $\rightarrow$  Age    (Nom, Prenom, Tel)  $\rightarrow$  Age , Tel**



## Autres axiomes

### Transitivité

☉ Si  $X \rightarrow Y$  et  $Y \rightarrow Z$  alors  $X \rightarrow Z$

### Union

☉ Si  $X \rightarrow Y$  et  $X \rightarrow Z$  alors  $X \rightarrow Y, Z$

### Pseudo-transitivité

☉ Si  $X \rightarrow Y$  et  $W, Y \rightarrow Z$  alors  $W, X \rightarrow Z$

### Décomposition

☉ Si  $X \rightarrow Y$  et  $Z \subseteq Y$  alors  $X \rightarrow Z$





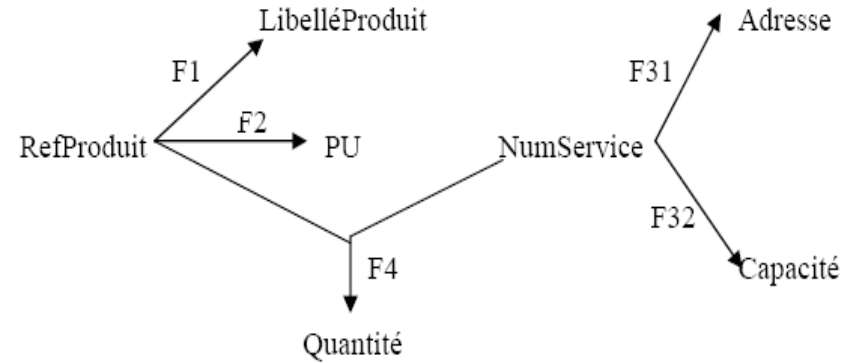
## Graphe de DFs

- Les DF peuvent être représentées à l'aide d'un graphe dont les nœuds sont les attributs impliqués dans les dépendances et les arcs représentent les dépendances elles-mêmes.
- Les arcs sont orientés de la partie gauche de la dépendance vers sa partie droite.
- L'origine d'un arc peut être multiple mais sa cible doit être un nœud unique.
- De ce fait, il est nécessaire d'avoir pour la construction d'un graphe de dépendance fonctionnelle un ensemble **canonique** de dépendances fonctionnelles.



## Exemple de graphe

- F1 : RefProduit → LibelléProduit
- F2 : RefProduit → PU
- F3 : NumService → Adresse, Capacité
- F4 : RefProduit, NumService → Quantité

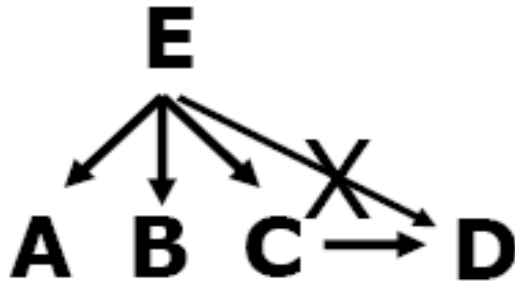


- La dépendance fonctionnelle F3 n'est pas canonique, il faut donc la décomposer en deux dépendances fonctionnelles F31 et F32
- F31 : NumService → Adresse
- F32 : NumService → Capacité



## Limites du graphe de DFs

- Vérifier que le graphe est bien **minimum**
- Trouver les identifiants de la relation (nœuds du graphe minimum à partir desquels on peut atteindre tous les autres nœuds (via les DF))
- Tester si la relation est bonne (bien normalisée)
- Sinon trouver les décompositions
- Exemple de graphe non minimum



$E \rightarrow D$  est déduite de  $E \rightarrow C$  et  $C \rightarrow D$   
Il faut supprimer  $E \rightarrow D$  du graphe



## Exemples d'application -1

Considérons le schéma de la relation suivante:  $R(A, B, C, D, E)$ . Cette relation est définie en extension par les tuples suivants :

| A  | B  | C  | D  | E  |
|----|----|----|----|----|
| a1 | b2 | c2 | d3 | e2 |
| a1 | b2 | c2 | d1 | e4 |
| a2 | b3 | c2 | d1 | e5 |
| a2 | b4 | c5 | d1 | e5 |

- Parmi les dépendances fonctionnelles suivantes, lesquelles s'appliquent à  $R$ ?
- Quelles sont les clés de  $R$ ?

$E \rightarrow D$     $D \rightarrow E$     $E \rightarrow B$     $E \rightarrow A$     $B \rightarrow C$     $B \rightarrow D$     $B \rightarrow A$

- NB : On peut définir une DF  $X \rightarrow Y$  si pour une valeur de  $X$ , on a toujours la même valeur de  $Y$ .



## Corrigé

1. Les dépendances fonctionnelles qui s'appliquent à R

$E \rightarrow D$

$E \rightarrow A$

$B \rightarrow C$

$B \rightarrow A$

2. Les clés de la relation R sont  $\{B, E\}$ .



## Exemples d'application -2

Considérons la relation suivante:

**PRODUIT** (RefProduit, LibelleProduit, PU, Quantité, NumService, Adresse, Capacité)

| RefProduit | LibelleProduit | PU     | Quantité | NumService | Adresse | Capacité |
|------------|----------------|--------|----------|------------|---------|----------|
| P1         | CH7            | 23.510 | 300      | S1         | Sousse  | 9000     |
| P1         | CH7            | 23.510 | 500      | S2         | Tunis   | 6000     |
| P3         | VIS12          | 0.150  | 900      | S4         | Sousse  | 2000     |

- Quelles sont les anomalies que représente cette relation ?
- Donner l'ensemble des dépendances fonctionnelles pour cette relation
- Comment corriger ces situations d'anomalie ?



# Pourquoi normaliser ?

- Ce bout de BD est un exemple de mauvaise conception de base de données:
  - La table montre de nombreuses redondances: les valeurs des attributs 'Num\_client' et 'adresse' contiennent des données multi-valeurs.
  - Les 2 clients sont situés dans le même département (Springfield). Ces informations ne sont pas séparées et la BD ne peut pas être facilement filtrée.

| Fact. N° | Date       | Client      | N° Client | Adresse                              | N° Inventaire | Produit         | N° Produit | No. | Prix    |
|----------|------------|-------------|-----------|--------------------------------------|---------------|-----------------|------------|-----|---------|
| 123      | 29/01/2018 | John Public | 11        | 35 Wood Lane, Springfield, ME 04487  | 1             | Monitor         | 2-0023-D   | 10  | \$200   |
| 123      | 29/01/2018 | John Public | 11        | 35 Wood Lane, Springfield, ME 04487  | 2             | Tapis de souris | 4-0023-D   | 12  | 50c     |
| 123      | 29/01/2018 | John Public | 11        | 35 Wood Lane, Springfield, ME 04487  | 3             | Chaise          | 5-0023-D   | 1   | \$120   |
| 124      | 30/01/2018 | Jane Doe    | 12        | 72 Windy Road, Springfield, ME 04487 | 1             | Ordinateur      | 1-0023-D   | 2   | \$1,200 |
| 124      | 30/01/2018 | Jane Doe    | 12        | 72 Windy Road, Springfield, ME 04487 | 2             | Casque          | 3-0023-D   | 2   | \$75    |

- On dit que c'est une BD **dénormalisée**: ne suit pas les règles de normalisation des BDs.
- Les données redondantes entraînent des anomalies sémantiques qui rendent le traitement automatique de la base de données et sa maintenance très difficile. En plus de l'augmentation des besoins en mémoire.
- **Solution**: Principe de **normalisation** et de l'application des **formes normales**.



# Principe de normalisation

- La normalisation est une stratégie et une approche de conception visant à éliminer les redondances dans les bases de données.
- La normalisation est donc le processus de transformation d'une relation ayant des problèmes lors de la mise à jour vers une autre relation n'ayant pas ces problèmes
- L'objectif est donc de définir un bon schéma relationnel qui décrit d'une manière convenable le système d'information d'une entreprise.
  - limiter les redondances de données,
  - limiter les pertes de données,
  - limiter les incohérences au sein des données
  - améliorer les performances des traitements.
- Afin de réaliser une bonne normalisation, trois formes normales séquentielles de base de données ont été développées pour les modèles de bases de données relationnelles.





# Première Forme Normale -1FN

- Une relation dans une BDR répond à la première forme normale (on dit qu'elle est en 1FN) lorsqu'elle remplit les critères suivants:
  - Toutes les données sont atomiques: un attribut ne peut prendre qu'une valeur et non un ensemble ou liste de valeur
  - Tous les champs de la relation contiennent des valeurs identiques

## Astuces:

- Si un attribut prend plusieurs valeurs == > créer une nouvelle relation ou diviser tous les données multi-valeurs en champs séparés.

Livre(numLivre,titre,auteurs) n'est pas en 1FN

auteurs : n'est pas un attribut atomique car peut avoir plusieurs valeurs

== >

Livre(numLivre,titre,#numAuteur)

Auteur(numauteur, nomAuteur)



## Exemple

- Pour revenir à l'exemple de facturation, les champs en rouge ne satisfont pas à l'une ou l'autre des exigences de la 1FN.

| Fact. N°. | Date       | Client      | N° Client | Adresse                              | N° inventaire | Produit         | N°-Produit | No. | Prix    |
|-----------|------------|-------------|-----------|--------------------------------------|---------------|-----------------|------------|-----|---------|
| 123       | 29/01/2018 | John Public | 11        | 35 Wood Lane, Springfield, ME 04487  | 1             | Monitor         | 2-0023-D   | 10  | \$200   |
| 123       | 29/01/2018 | John Public | 11        | 35 Wood Lane, Springfield, ME 04487  | 2             | Tapis de souris | 4-0023-D   | 12  | 50c     |
| 123       | 29/01/2018 | John Public | 11        | 35 Wood Lane, Springfield, ME 04487  | 3             | Chaise          | 5-0023-D   | 1   | \$120   |
| 124       | 30/01/2018 | Jane Doe    | 12        | 72 Windy Road, Springfield, ME 04487 | 1             | Ordinateur      | 1-0023-D   | 2   | \$1,200 |
| 124       | 30/01/2018 | Jane Doe    | 12        | 72 Windy Road, Springfield, ME 04487 | 2             | Casque          | 3-0023-D   | 2   | \$75    |



| Fact. N°. | Date       | Prénom | Nom    | N° Client | Adresse Postale | Ville       | Etat | Code Postal | N° inventaire | Produit         | N°-Produit | No. | Prix en € |
|-----------|------------|--------|--------|-----------|-----------------|-------------|------|-------------|---------------|-----------------|------------|-----|-----------|
| 123       | 01/29/2018 | John   | Public | 11        | 35 Wood Lane    | Springfield | ME   | 04487       | 1             | Monitor         | 2-0023-D   | 10  | 200       |
| 123       | 01/29/2018 | John   | Public | 11        | 35 Wood Lane    | Springfield | ME   | 04487       | 2             | Tapis de souris | 4-0023-D   | 12  | 0.50      |
| 123       | 01/29/2018 | John   | Public | 11        | 35 Wood Lane    | Springfield | ME   | 04487       | 3             | Chaise          | 5-0023-D   | 1   | 120       |
| 124       | 01/30/2018 | Jane   | Doe    | 12        | 72 Windy Road   | Springfield | ME   | 04487       | 1             | Ordinateur      | 1-0023-D   | 2   | 1200      |
| 124       | 01/30/2018 | Jane   | Doe    | 12        | 72 Windy Road   | Springfield | ME   | 04487       | 2             | Casque          | 3-0023-D   | 2   | 75        |

**Solution:** les champs client et adresse sont divisées en attributs plus spécifiques

- Client devient: nom et prénom (ou carrément une autre relation à part)
- Adresse devient: adresse municipale, ville, état, code postal...
- Adopter une même devise pour la colonne prix: convertir les Cents en dollars.



## Deuxième Forme Normale -2FN

- Une relation dans une BDR répond à la deuxième forme normale (on dit qu'elle est en 2FN) lorsqu'elle remplit les critères suivants:
  - Elle est en 1FN
  - si la clé primaire est composée de plusieurs attributs, les autres attributs de la relation doivent dépendre de la totalité de la clé et non d'une partie

### Astuce:

- déplacer tous les attributs non-clés qui n'ont pas de dépendance fonctionnelle avec la totalité de la clé primaire dans des relations séparées.

la relation `Commande(num_Client, codeProduit, nomClient, designationProduit)`  
n'est pas en 2FN car :  
DF: `num_Client`  $\rightarrow$  `nomClient` et `codeProduit`  $\rightarrow$  `designationProduit`

### Correction :

`Client(num_Client, nom)`  
`Article(codeProduit, designationProduit)`  
`Commande(#num_Client, #codeArticle)`



## Exemple

- Pour revenir à l'exemple de facturation, une clé primaire possible résulte de: numéro de facture, numéro de client et numéro inventaire. Par exemple, les valeurs {124,12,1} permet de désigner l'achat de l'ordinateur portable de Jane.
- Cependant, le code postal n'a de dépendance fonctionnelle qu'avec le numéro de client. Aussi, la date dépend uniquement du numéro de la facture.
- **Solution:** tous les attributs entièrement dépendants du numéro de facture ont été déplacées vers une relation 'Facture'.

| Fact. N°. | N° Client | N° inventaire | Date       | Prénom | Nom    | Adresse Postale | Ville       | Etat | Code Postal | Produit         | N° Produit | No. | Prix en € |
|-----------|-----------|---------------|------------|--------|--------|-----------------|-------------|------|-------------|-----------------|------------|-----|-----------|
| 123       | 11        | 1             | 29/01/2018 | John   | Public | 35 Wood Lane    | Springfield | ME   | 04487       | Monitor         | 2-0023-D   | 10  | 200       |
| 123       | 11        | 2             | 29/01/2018 | John   | Public | 35 Wood Lane    | Springfield | ME   | 04487       | Tapis de souris | 4-0023-D   | 12  | 0.50      |
| 123       | 11        | 3             | 29/01/2018 | John   | Public | 35 Wood Lane    | Springfield | ME   | 04487       | Chaise          | 5-0023-D   | 1   | 120       |
| 124       | 12        | 1             | 30/01/2018 | Jane   | Doe    | 72 Windy Road   | Springfield | ME   | 04487       | Ordinateur      | 1-0023-D   | 2   | 1200      |
| 124       | 12        | 2             | 30/01/2018 | Jane   | Doe    | 72 Windy Road   | Springfield | ME   | 04487       | Casque          | 3-0023-D   | 2   | 75        |



| facture   |            |        |        |           |                 |             |      |             |
|-----------|------------|--------|--------|-----------|-----------------|-------------|------|-------------|
| Fact. N°. | Date       | Prénom | Nom    | N° Client | Adresse Postale | ville       | Etat | Code postal |
| 123       | 29/01/2018 | John   | Public | 11        | 35 Wood Lane    | Springfield | ME   | 04487       |
| 124       | 30/01/2018 | Jane   | Doe    | 12        | 72 Windy Road   | Springfield | ME   | 04487       |

| Poste de facture |               |                 |            |     |           |
|------------------|---------------|-----------------|------------|-----|-----------|
| Fact. N°.        | N° inventaire | Produit         | N° Produit | No. | Prix en € |
| 123              | 1             | Monitor         | 2-0023-D   | 10  | 200       |
| 123              | 2             | Tapis de souris | 4-0023-D   | 12  | 0.50      |
| 123              | 3             | Chaise          | 5-0023-D   | 1   | 120       |
| 124              | 1             | Ordinateur      | 1-0023-D   | 2   | 1200      |
| 124              | 2             | Casque          | 3-0023-D   | 2   | 75        |



## Troisième Forme Normale -3FN

- Une relation dans une BDR répond à la troisième forme normale (on dit qu'elle est en 3FN) lorsqu'elle remplit les critères suivants:
  - Elle est en 2FN
  - Tous les attributs qui n'appartiennent pas à l'identifiant (clé primaire) doivent dépendre directement de l'identifiant et d'aucun autre attribut
- **Astuce:** placer l'attribut pathologique dans une relation séparée.

Avions(NumAvion, type, capacite, poids, propriétaire) n'est pas en 3FN car :

NumAvion → propriétaire  
type → capacite, poids

Correction :

Avions(NumAvion, #type, propriétaire)  
Type(type, capacité, poids)



## Exemple -1

- Pour revenir à l'exemple de facturation, dans la relation "Facture", le nom et le prénom, l'adresse, la ville, l'état et le code postal dépendent non seulement de la clé primaire (le numéro de facture), mais aussi du numéro du client.
- Dans la relation "Poste de facture", les champs produits et prix dépendent non seulement de la clé primaire, mais aussi du numéro de produit.

| Facture         |            |        |        |           |                 |             |      |             |
|-----------------|------------|--------|--------|-----------|-----------------|-------------|------|-------------|
| <u>Fact. N°</u> | Date       | Prénom | Nom    | N° Client | Adresse Postale | Ville       | Etat | Code postal |
| 123             | 29/01/2018 | John   | Public | 11        | 35 Wood Lane    | Springfield | ME   | 04487       |
| 124             | 29/01/2018 | Jane   | Doe    | 12        | 72 Windy Road   | Springfield | ME   | 04487       |

| Poste de Facture |                      |          |            |     |           |
|------------------|----------------------|----------|------------|-----|-----------|
| <u>Fact. N°</u>  | <u>N° inventaire</u> | Produit  | N° Produit | No. | Prix en € |
| 123              | 1                    | Monitor  | 2-0023-D   | 10  | 200       |
| 123              | 2                    | Mousepad | 4-0023-D   | 12  | 0.50      |
| 123              | 3                    | Chair    | 5-0023-D   | 1   | 120       |
| 124              | 1                    | Laptop   | 1-0023-D   | 2   | 1200      |
| 124              | 2                    | Headset  | 3-0023-D   | 2   | 75        |



## Exemple -2

Solution:

- déplacer les champs dans des tables séparées, reliées entre elles par des clés étrangères. Il en résulte les quatre tableaux normalisés : "Facture", "Client", "Poste de Facture" et "Produit".

| Facture         |            |           |
|-----------------|------------|-----------|
| <u>Fact. N°</u> | Date       | N° Client |
| 123             | 01/29/2018 | 11        |
| 124             | 01/30/2018 | 12        |

| Client           |        |        |                 |             |      |             |
|------------------|--------|--------|-----------------|-------------|------|-------------|
| <u>N° Client</u> | Prénom | Nom    | Adresse Postale | Ville       | Etat | Code Postal |
| 11               | John   | Public | 35 Wood Lane    | Springfield | ME   | 04487       |
| 12               | Jane   | Doe    | 72 Windy Road   | Springfield | ME   | 04487       |

| Produits          |                 |           |
|-------------------|-----------------|-----------|
| <u>N° Produit</u> | Produit         | Prix en € |
| 1-0023-D          | Ordinateur      | 1200      |
| 2-0023-D          | Monitor         | 200       |
| 3-0023-D          | Casque          | 75        |
| 4-0023-D          | Tapis de souris | 0.50      |
| 5-0023-D          | Chaise          | 120       |

| Poste de facture |                      |            |     |
|------------------|----------------------|------------|-----|
| <u>Fact. N°</u>  | <u>N° Inventaire</u> | N° Produit | No. |
| 123              | 1                    | 2-0023-D   | 10  |
| 123              | 2                    | 4-0023-D   | 12  |
| 123              | 3                    | 5-0023-D   | 1   |
| 124              | 1                    | 1-0023-D   | 2   |
| 124              | 2                    | 3-0023-D   | 2   |

5

## Le langage SQL





# Présentation générale

- SQL : Structured Query Language: Permet de communiquer avec un SGBDR
- Présente les avantages suivants :
  - langage efficace pour la communication avec une SGBDR
  - facile à apprendre et à utiliser
  - dispose de fonctionnalités complètes : définir, extraire et manipuler les données des tables
- SQL ne fait pas la différence entre les lettres minuscules et majuscules (sauf après la condition **where attribut=valeur** et les voyelles accentuées ne sont pas acceptées.

## Interrogation de données :

**SELECT** : Extraction de données

## Langage de manipulation de (LMD) :

**INSERT** : insertion

**UPDATE** : mise à jour

**DELET** : suppression de lignes

## Langage de définition de données (LDD) :

**CREATE** : création d'un objet (table, ...)

**ALTER** : modification d'un objet

**DROP** : supprimer un objet

**RENAME** : : renommer un objet



# Interrogation des données

- Toute interrogation se fait par la commande « **SELECT** »
- Commande déclarative : décrit ce que l'on cherche sans décrire le moyen de le réaliser
- l'instruction SELECT doit inclure :
  - une clause SELECT qui détermine les **colonnes** à afficher
  - une clause FROM : détermine la **table** contenant les colonnes répertoriées dans la clause SELECT
- Le nom de la table peut être omis quand il n'y a pas d'ambiguïté. Il doit être précisé s'il y a une ambiguïté, ce qui peut arriver quand on fait une sélection sur plusieurs tables à la fois et que celles-ci contiennent des colonnes qui ont le même nom

```
SELECT [DISTINCT] {*} column [alias] | expression1 [as alias] ...}  
FROM table1  
[WHERE condition (s) ]  
[GROUP BY expression]  
[HAVING condition]  
[ORDER BY {column, expr, alias} [ASC|DESC]]  
[{UNION|INTERSECT|MINUS} (sous requête)] ;
```

*Notes :*

| : choix entre différentes options  
{ } : choix obligatoire  
[] : facultatif



# Duplicatas des données



la directive **DISTINCT** élimine les éventuels duplicatas

| brevet | nom              | nbrHvol | prime | embauche   | typeAvion | compa |
|--------|------------------|---------|-------|------------|-----------|-------|
| PL-1   | Gratien Viel     | 450     | 500   | 05/02/1965 | A320      | AF    |
| PL-2   | Didier Donsez    | 0       |       | 13/05/1965 | A320      | AF    |
| PL-3   | Richard Grin     | 1000    |       | 11/09/2001 | A320      | SING  |
| PL-4   | Placide Fresnais | 2450    | 500   | 21/09/2001 | 1330      | SING  |
| PL-5   | Daniel Vielle    | 400     | 600   | 16/01/1965 | A340      | AF    |
| PL-6   | Françoise Tort   |         | 0     | 24/12/2000 | A340      | CAST  |

SELECT compa FROM Pilote

Compa

AF  
AF  
SING  
SING  
AF  
CAST

Avec  
duplicatas

SELECT **DISTINCT** compa FROM Pilote

Compa

AF  
SING  
AF  
CAST

Sans  
duplicatas



## Alias des données

- **ALIAS** permet de renommer des colonnes à l'affichage ou des tables dans la requête: applicable pour les colonnes et les tables

Soit la relation : employe(nom,num,fonction,n\_sup,embauche,salaire,comm,dept)

salaire mensuelle de chaque salarié :

```
SELECT nom AS nomPersonne, sal*12 AS Salaire_Annuel FROM employe
```

- Si on définit l'alias d'une table, il faut préfixer les colonnes par l'alias de la table

Exemple:

```
SELECT emp.nom FROM employe emp;
```



# WHERE pour la restriction -1



Instruction **WHERE** suivie d'un **prédicat**: comparaison de deux expressions ou plus au moyen d'un opérateur

## Opérateurs logiques : AND et OR

- Peuvent être utilisés pour combiner plusieurs prédicats
- L'opérateur **AND** est prioritaire par rapport à l'opérateur **OR**
- L'opérateur **NOT** placé devant un prédicat en inverse le sens.

**SELECT ... WHERE condition1 AND/OR condition2**

```
WHERE exp1 = exp2
WHERE exp1 != exp2
WHERE exp1 < exp2
WHERE exp1 > exp2
WHERE exp1 <= exp2
WHERE exp1 >= exp2
WHERE exp1 BETWEEN exp2 AND exp3
WHERE exp1 LIKE exp2
WHERE exp1 NOT LIKE exp2
WHERE exp1 IN (exp2, exp3,...)
WHERE exp1 NOT IN (exp2, exp3,...)
WHERE exp IS NULL
WHERE exp IS NOT NULL
```



## WHERE pour la restriction -2

### Exemple:

soit la relation: **Emp(empno,ENAME job,embauche,sal,tel,#deptno)**

1. afficher le nom de tous les employés dont la deuxième lettre du nom est un 'a' et qui ont été embauchés en 1987
2. afficher le nom et date d'embauche de tous les employés qui ont été embauché depuis 2000.
3. Salaire et nom des employés gagnant entre 15000 et 18000
4. Sélectionner les employés du département 30 ayant un salaire supérieur à 1500
5. Noms des directeurs et des administratifs du département 30
6. Donnez la liste des employés dont le numéro de téléphone n'est pas renseigné



## WHERE pour la restriction -3

### Corrigé:

soit la relation: **Emp(empno,ename job,embauche,sal, tel,#deptno)**

1. SELECT ename FROM Emp WHERE ename LIKE '\_a%' AND embauche = 1987;
2. SELECT ename, embauche FROM Emp WHERE embauche >= 2000;
3. SELECT sal, ename FROM Emp WHERE sal BETWEEN 15000 AND 18000;
4. SELECT \* FROM Emp WHERE deptno = 30 AND sal > 1500;
5. SELECT ename FROM Emp WHERE deptno = 30 AND job IN ('DIRECTEUR', 'ADMINISTRATIF');
6. SELECT ename FROM Emp WHERE tel IS NULL



## ORDER BY pour l'ordonnancement

● **Clause ORDER BY**: Classement des résultats d'un SELECT et précise l'ordre dans lequel la liste des lignes sélectionnées sera donnée

● On peut lui associer **ASC** ou **DESC**

- ASC: Ordre croissant
- DESC: ordre décroissant

Exemple : Donner pour chaque fonction la liste des noms des employés ainsi que leurs salaire trié dans l'ordre décroissant

```
SELECT ename, job, sal  
FROM emp  
ORDER BY job, sal DESC
```



| Ename    | job           | sal   |
|----------|---------------|-------|
| BARA     | administratif | 15000 |
| BALIN    | administratif | 13500 |
| LAMBERT  | administratif | 12000 |
| DUPONT   | administratif | 9000  |
| DUPOND   | commercial    | 25000 |
| LEFEBVRE | commercial    | 23500 |
| PAQUEL   | commercial    | 22000 |
| MARTIN   | commercial    | 20000 |
| LEBRETON | commercial    | 15000 |





## Fonctions de groupes -1

- Permettent de faire des calculs sur l'ensemble des valeurs d'une colonne
- Les fonctions de groupes peuvent apparaître dans le Select ou le Having. Ci-dessous quelques fonctions usuelles

AVG moyenne  
SUM somme  
MIN plus petite des valeurs  
MAX plus grande des valeurs  
VARIANCE variance  
STDDEV écart type (déviation standard)  
COUNT(\*) nombre de lignes  
COUNT(col ) nombre de valeurs non nulles de la colonne  
COUNT(DISTINCT col ) nombre de valeurs non nulles différentes



## Fonctions de groupes -2

Soit la table Pilote ci-dessous

1. Moyenne des heures de vol et des primes des pilotes de la compagnie 'AF'
2. Nombre de pilotes, nombre total d'heures effectuées par tous les pilotes, nombre de primes (toutes et distinctes)
3. le nombre d'heures de vol le plus élevé, le nombre d'heure de vol le moins élevé
4. date d'embauche la plus récente, date d'embauche la plus ancienne, la prime la plus faible, la prime la plus élevée, la moyenne des primes

| brevet | nom              | nbrHvol | prime | embauche   | typeAvion | compa |
|--------|------------------|---------|-------|------------|-----------|-------|
| PL-1   | Gratien Viel     | 450     | 500   | 05/02/1965 | A320      | AF    |
| PL-2   | Didier Donsez    | 0       |       | 13/05/1965 | A320      | AF    |
| PL-3   | Richard Grin     | 1000    |       | 11/09/2001 | A320      | SING  |
| PL-4   | Placide Fresnais | 2450    | 500   | 21/09/2001 | 1330      | SING  |
| PL-5   | Daniel Vielle    | 400     | 600   | 16/01/1965 | A340      | AF    |
| PL-6   | Françoise Tort   |         | 0     | 24/12/2000 | A340      | CAST  |



## Fonctions de groupes -3

### Corrigé

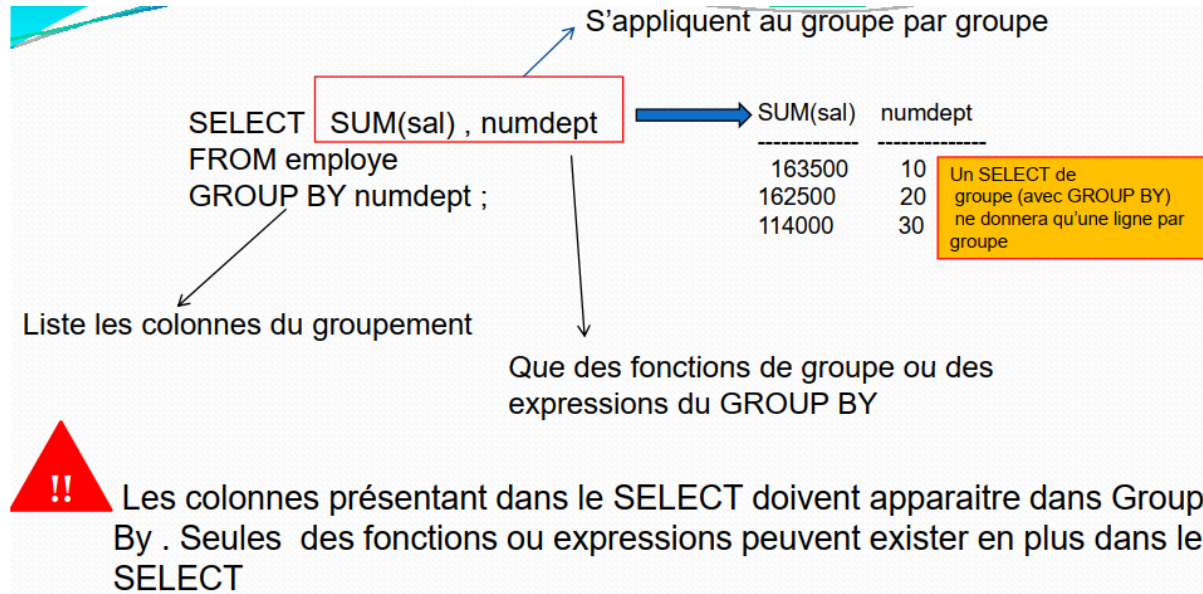
1. `SELECT AVG(nbrHvol) AS moy_heures, AVG(prime) AS moy_primes FROM Pilote WHERE compa = 'AF';`
2. `SELECT COUNT(*) AS nb_pilotes, SUM(nbrHvol) AS total_heures, COUNT(prime) AS nb_primes_reseignees, COUNT(DISTINCT prime) AS nb_primes_distinctes FROM Pilote;`
3. `SELECT MAX(nbrHvol) AS heures_max, MIN(nbrHvol) AS heures_min FROM Pilote;`
4. `SELECT MAX(embauche) AS embauche_plus_recente, MIN(embauche) AS embauche_plus_ancienne, MIN(prime) AS prime_min, MAX(prime) AS prime_max, AVG(prime) AS prime_Moyenne FROM Pilote;`

**Remarque: AVG et COUNT ignorent automatiquement les valeurs non renseignées (NULL)**



# GROUP BY pour le regroupement -1

- **Clause GROUP BY:** constituer des groupements de lignes et liste les colonnes du groupement
- == > Un SELECT de groupe (avec GROUP BY) ne donnera qu'une ligne par groupe





## GROUP BY pour le regroupement -2

- Utilisation de la clause GROUP BY expr1, expr2, ... : Parmi les lignes ayant même valeur pour expr1, on regroupe celle ayant même valeur pour expr2, ... (voir plus loin)

Exemple :

Nombre de pilote qualifiés par compagnie et par type d'appareil

```
== > SELECT compa, typeAvion, COUNT(brevet) FROM Pilote  
GROUP BY compa, typeAvion;
```



| compa | typeAvion | COUNT(brevet) |
|-------|-----------|---------------|
| AF    | A320      | 2             |
| AF    | A340      | 1             |
| CAST  | A340      | 1             |
| SING  | A320      | 1             |
| SING  | A330      | 1             |

Que des fonctions de groupe ou des expressions du GROUP BY



## GROUP BY pour le regroupement -3

● **Clause HAVING** joue le rôle de filtre pour la clause GROUP BY, mais ne porte que sur des caractéristiques de groupe :

- \*) fonction de groupe
- \*) expressions présentes dans le GROUP BY

```
SELECT col1, fonction1Groupe, ...  
FROM table (s)  
[WHERE condition (s)]  
[GROUP BY cole1[,col2, ...]]  
[HAVING condition]  
;
```

Permet d'exclure des lignes pour chaque groupement ou de rejeter des groupements entiers. Elle s'applique à la totalité de la table

liste les colonnes du groupement

Permet de poser des conditions sur chaque groupement



## GROUP BY pour le regroupement -4



### Exemple Clause HAVING

Exemple : Liste des salaires moyens par fonction (job) ayant plus de deux employés.

```
SELECT fonction, COUNT(*), AVG(sal)
FROM employe
GROUP BY fonction
HAVING COUNT(*) >= 2;
```



## Exercice

⦿ soit la relation : **Pilote(brevet,nom,nbHvol,typeavion,compa)**

Ecrire les requêtes répondant aux questions suivantes :

1. Moyenne des heures de vol et des primes par compagnie
2. Nombre de pilote par compagnie
3. Nombre d'heures de vol le plus élevé, date d'embauche la plus récente par compagnie
4. somme des heures de vol des pilotes volant sur A320 par compagnie
5. Nombre de pilotes qualifié (nombre de brevet) par compagnie et par type d'appareil
6. le nombre de pilotes par compagnie ayant plus d'un pilote





## Corrigé -1



soit la relation : **Pilote(brevet,nom,nbHvol,typeavion,compa)**

Moyenne des heures de vol et des primes par compagnie

```
SELECT compa, AVG(nbHvol), AVG(prime) FROM Pilote GROUP By compa;
```

| Compa | AVG(nbHvol) | AVG(prime) |
|-------|-------------|------------|
| AF    | 283,33      | 550        |
| CAST  |             | 0          |
| SING  | 1725        | 500        |

Nombre de pilotes par compagnie

```
SELECT compa, count(*) From Pilote GROUP By compa;
```

| Compa | COUNT(*) |
|-------|----------|
| AF    | 3        |
| CAST  | 1        |
| SING  | 2        |



## Corrigé -2

soit la relation : **Pilote(brevet,nom,nbHvol,typeavion,compa)**

Nombre d'heures de vol le plus élevé, date d'embauche la plus récente par compagnie:

```
SELECT compa, MAX(nbHvol), MAX(embauche) as 'date embauche recente' FROM Pilote GROUP By compa;
```

| Compa | MAX(nbHvol) | date embauche recente |
|-------|-------------|-----------------------|
| AF    | 450         | 13/05/95              |
| CAST  |             | 24/12/00              |
| SING  | 2450        | 21/09/01              |

somme des heures de vol des pilotes volant sur A320 par compagnie

```
SELECT compa, SUM(nbHvol) From Pilote WHERE typAvion='A320' GROUP By compa;
```

| Compa | SUM(NBHVOL) |
|-------|-------------|
| AF    | 450         |
| SING  | 1000        |



## Corrigé -3

Nombre de pilotes qualifié (nombre de brevet) par compagnie et par type d'appareil

```
SELECT compa, typAvion, count(brevet) FROM Pilote GROUP By compa, typeAvion;
```

| Compa | typeAvion | count(brevet) |
|-------|-----------|---------------|
| AF    | A320      | 2             |
| AF    | A340      | 1             |
| CAST  | A340      | 1             |
| SING  | A320      | 1             |
| SING  | A330      | 1             |

le nombre de pilotes par compagnie ayant plus d'un pilote

```
SELECT compa, count(brevet) Frome Pilote GROUP By compa HAVING count(brevet)>=2;
```

| Compa | COUNT(brevet) |
|-------|---------------|
| AF    | 3             |
| SING  | 2             |



# Jointure

-1

- Les jointures permettent d'extraire les données issues de plusieurs tables  
== > met en relation deux tables sur la base d'une condition de jointure (comparaison de colonne)
- Tout simplement en spécifiant plusieurs tables après le FROM
- Une condition de jointure exprime une relation existante entre les données d'une colonne d'une table et de celle d'une autre colonne d'une autre table  
== > généralement fait intervenir une clé étrangère d'une table avec une clé primaire d'une autre table



## Joiture

-2

- La clé secondaire de la table emp correspond à la clé primaire de dep

Table emp

| num   | nom     | fonction      | num_sup | embauche  | sal   | comm | numdept |
|-------|---------|---------------|---------|-----------|-------|------|---------|
| 16712 | MARTIN  | directeur     | 25717   | 23-MAY-90 | 40000 |      | 30      |
| 17574 | DUPONT  | administratif | 16712   | 03-MAY-95 | 9000  |      | 30      |
| 26691 | DUPOND  | commercial    | 27047   | 04-APR-88 | 25000 | 2500 | 20      |
| 25012 | LAMBERT | administratif | 27047   | 14-APR-91 | 12000 |      | 20      |
| ...   |         |               |         |           |       |      |         |

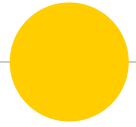
Table dept

| numdept | Nomd        | lieu   |
|---------|-------------|--------|
| 10      | Recherche   | Rennes |
| 20      | vente       | Metz   |
| 30      | direction   | Gif    |
| 40      | fabrication | Reims  |

Exemple : liste des employés et de leur lieux de travail

```
SELECT emp.nom, lieu  
FROM emp, dept  
WHERE emp.numdept = dept.numdept ;
```

Jointure entre tables  
« emp » et « dept »



## **Fin du chapitre 1**